



Shift-T: Universal Transpilation Motor (UTM)

Moviendo la 'T' del ETL al ELT: De Legacy a Lakehouse.

Migración Inteligente • Arquitectura de Agentes • Control Total

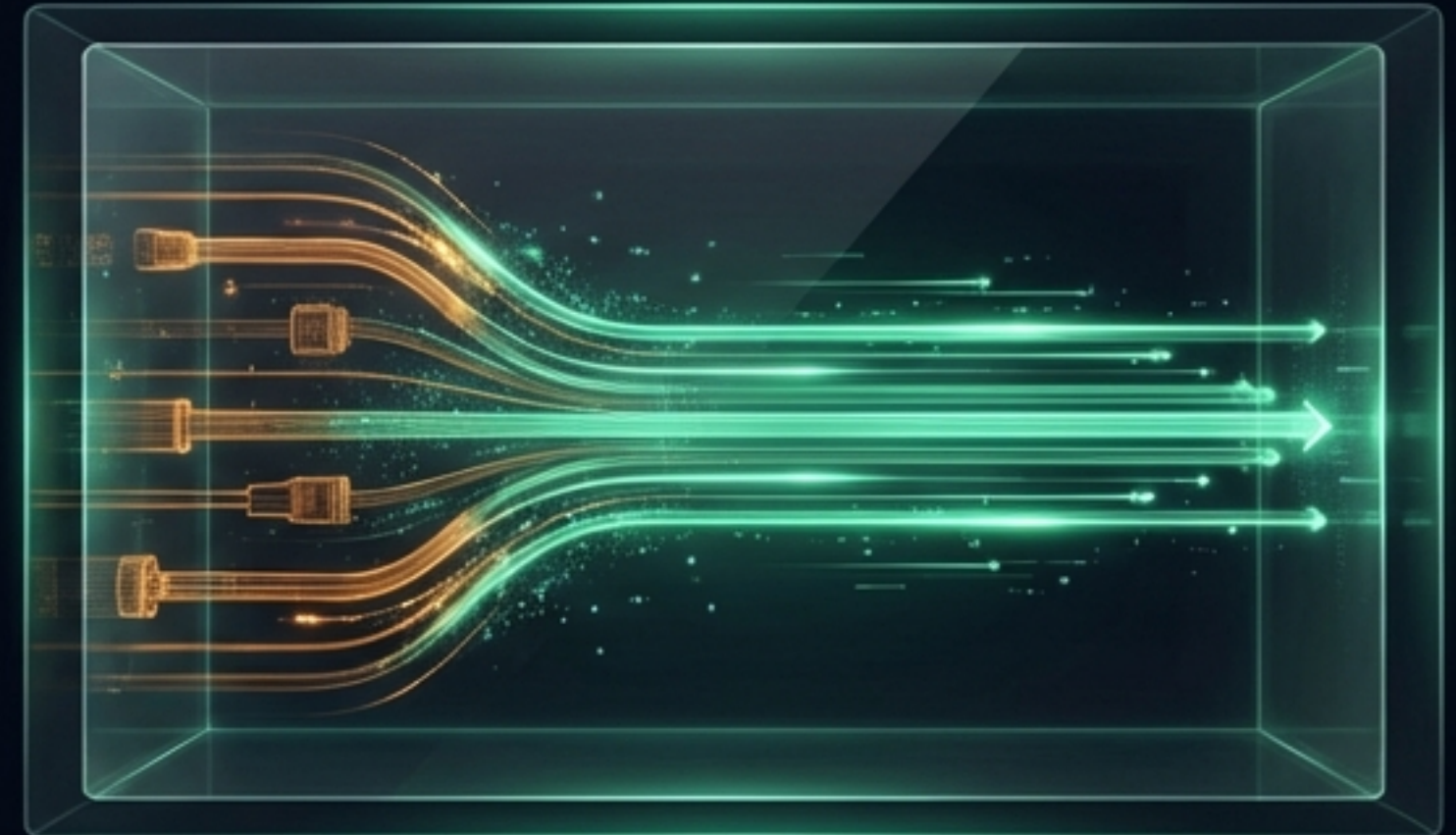
El desafío no es traducir código, es modernizar la arquitectura

El Mundo Legacy (ETL)



- Procesos rígidos (SSIS, SAP IQ, Informatica)
- Lógica procedimental ('Spaghetti code')
- Procesamiento en tránsito que estrangula el rendimiento

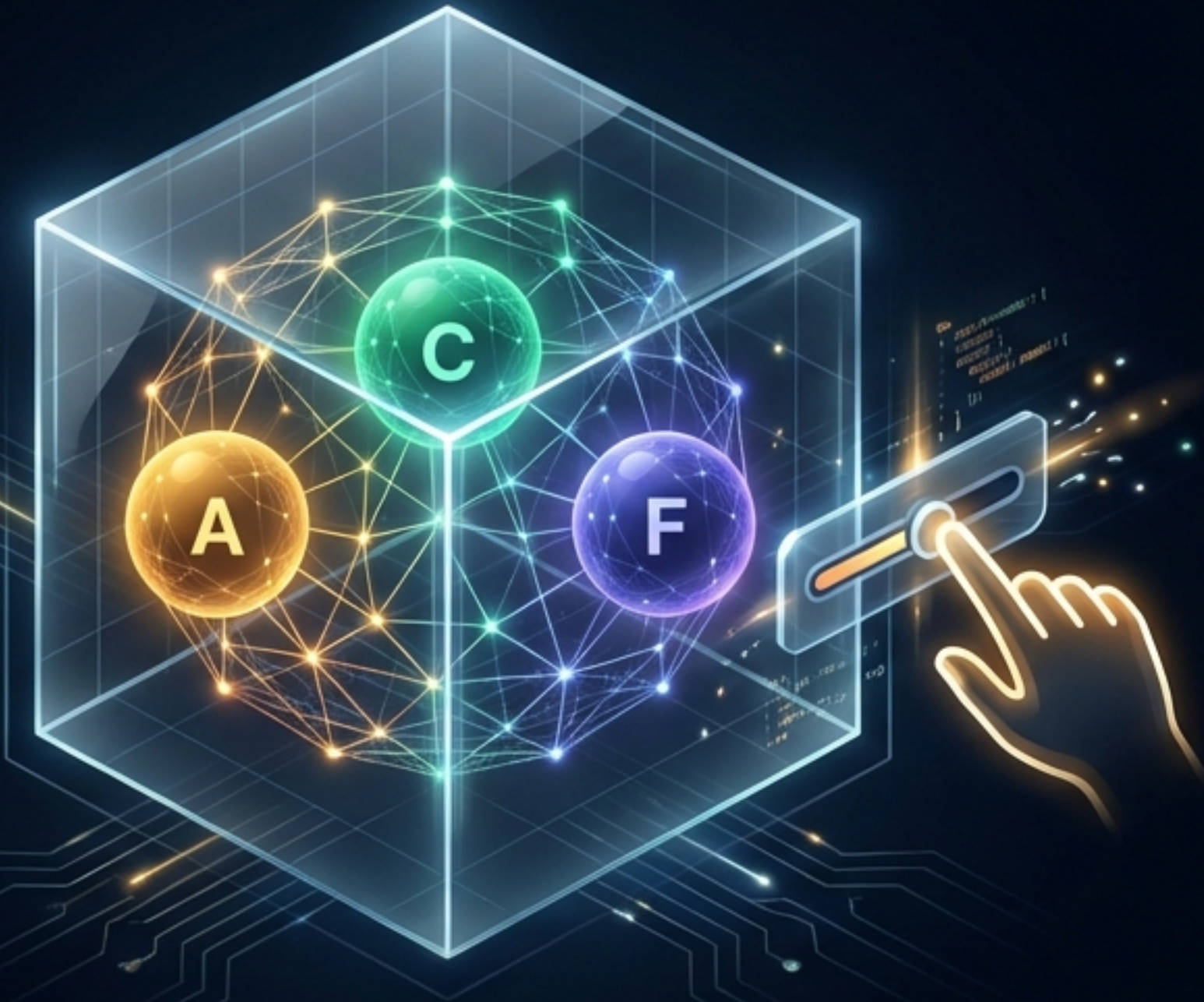
El Futuro (ELT)



- Ecosistemas nativos en Cloud (Databricks, Snowflake)
- Lógica vectorizada y funcional (PySpark)
- Procesamiento en destino escalable

Shift-T no hace un 'Lift & Shift'. Entiende el contexto, el linaje y la intención del negocio para transformar verdaderamente la arquitectura.

Una plataforma de "Caja de Cristal" impulsada por Inteligencia Artificial



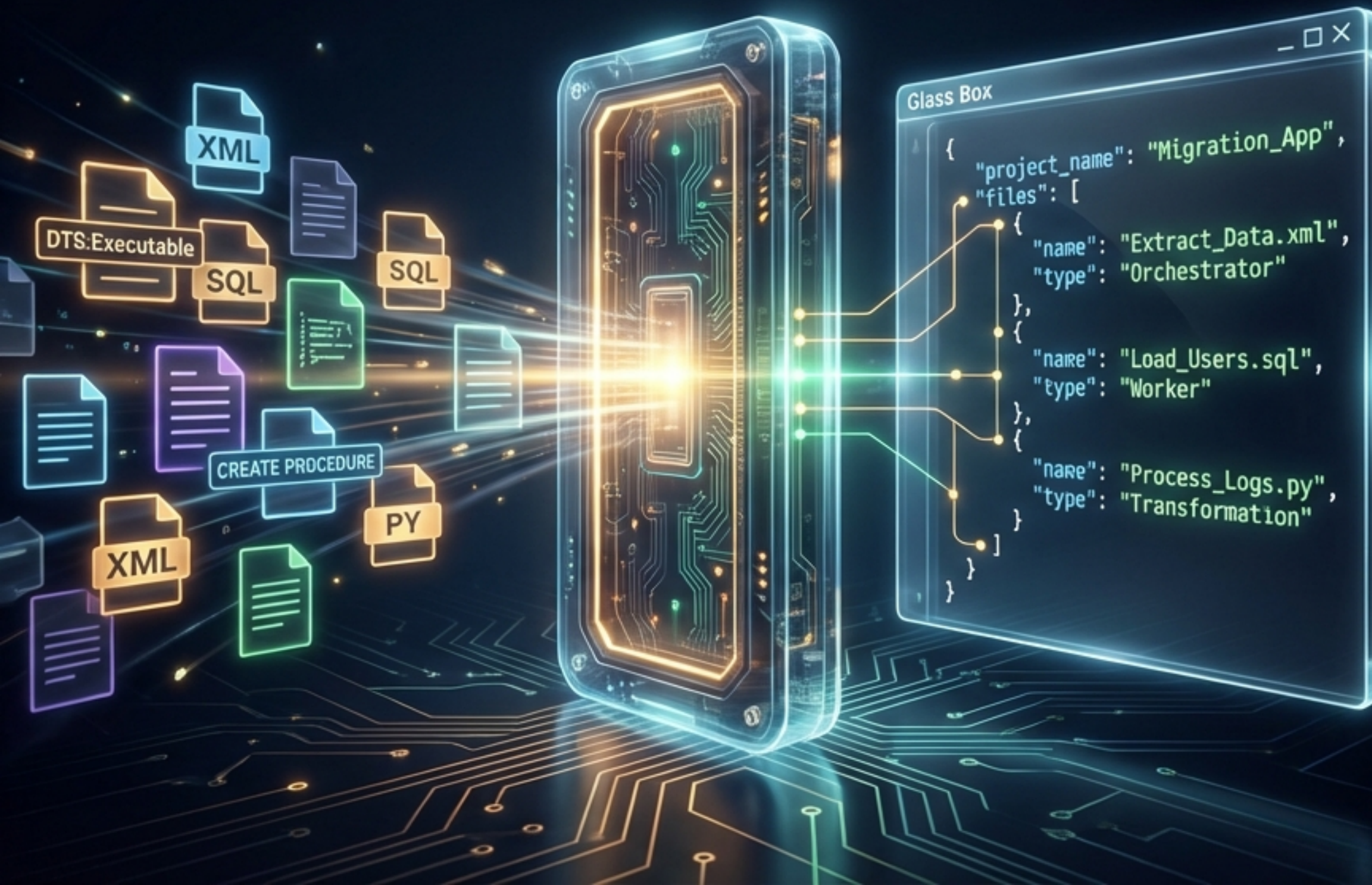
- **Sistema Multi-Agente**
Un equipo de agentes especializados (Arquitecto, Intérprete, Crítico) orquestados por LangGraph.
- **User-Centric**
El usuario valida, corrige y guía a la IA. Nada ocurre sin supervisión.
- **Agnosticismo Tecnológico**
Ingesta: SSIS, PL/SQL, SAP IQ.
Salida: PySpark, Iceberg, SQL nativo.
- **Tech Stack**
Python, FastAPI y Supabase para persistencia de estado y linaje.

El Plano Operativo: 4 Estadios de Transformación



Estadio 1: Triage y el 'Scanner' de Ingeniería Inversa

Del caos de archivos a un inventario estructurado.



1. Ingesta

- Conexión directa a repositorio GitHub o carga de ZIP.

2. Fingerprinting

- Detección de firmas y "Magic Strings" (ej: DTS:Executable, CREATE PROCEDURE).

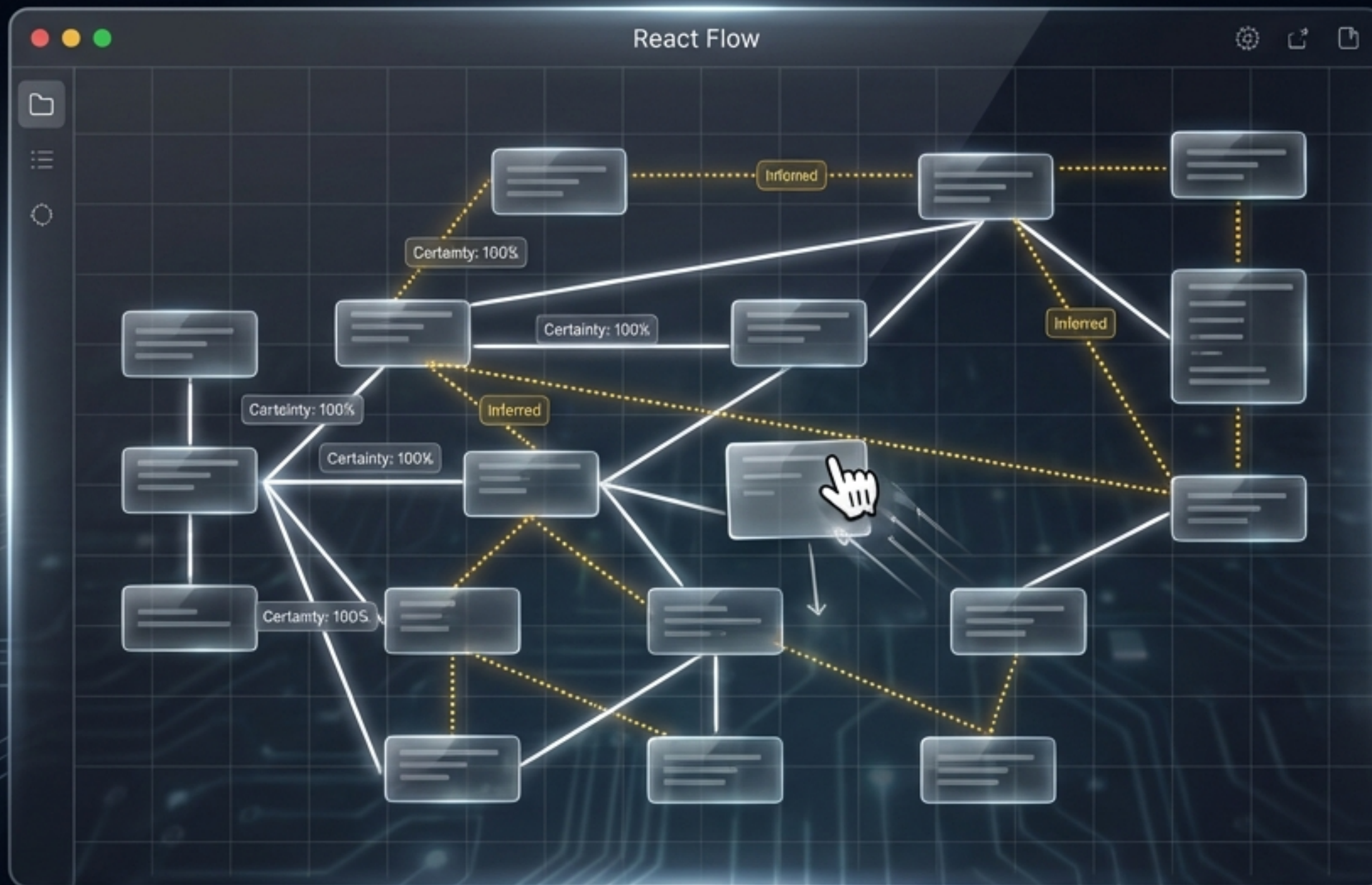
3. Clasificación

- Separación automática: Orquestadores vs. Workers vs. Transformaciones.

4. Resultado

- Un "Manifiesto de Estructura" limpio que elimina el ruido antes de consumir tokens de IA.

Agente A: El Arquitecto de la Malla



Rol del Agente:

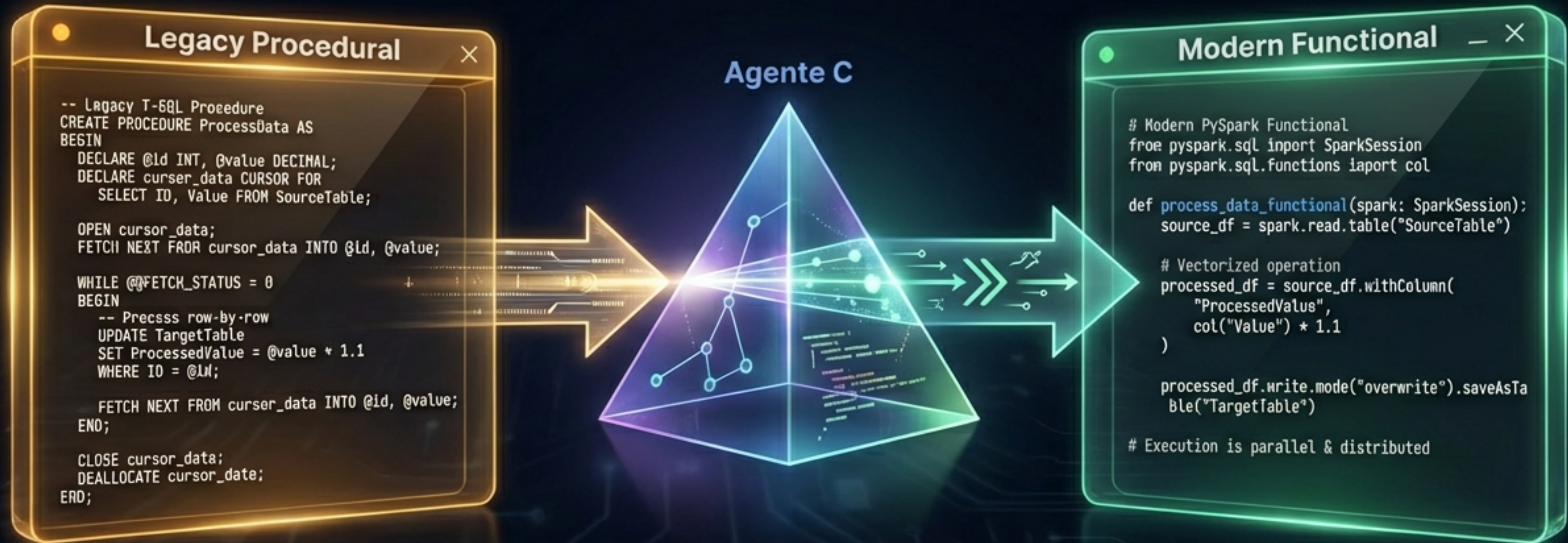
Agente A analiza dependencias ocultas (`ExecutePackageTask`, llamadas externas) para construir la topología.

Interacción (Drag & Drop):

- Visualización de la orquestación detectada.
- **Interacción Real**: Arrastrar cajas y reconectar flechas.
- **Validación**: El usuario aprueba 'El Plano' antes de generar código.

Estadio 2: Transpilación Semántica (Agente C)

De lógica procedimental a ejecución funcional distribuida.



El Intérprete (Agente C): No traduce línea por línea; entiende la intención del negocio.

Cambio de Paradigma: Convierte bucles y cursores secuenciales en operaciones vectorizadas de PySpark (paralelas).

Generación de Draft: Primera versión funcional manteniendo la estructura lógica original.

Estadio 3: El Ciclo de Refinamiento (Agente F)



Rol del Agente F:

Actúa como un auditor senior. Revisa el código generado por el Agente C en busca de ineficiencias y antipatronos.

Ejemplos de Optimización:

- Seguridad: **Detección de riesgos.**
- Arquitectura: "¿Podemos **paralelizar** esta carga?"
- Estándares: "Implementar **Arquitectura Medallón** (Bronce/Plata/Oro)"

Control Total y Personalización del Motor

 **Shift-T**

Model Selector: Claude 3.5 Sonnet (Code) / GPT-4o (Analysis) ▾

Original SQL

```
1 - -- Legacy T-SQL Procedure
2 CREATE PROCEDURE ProcessData AS
3 BEGIN
4 - DECLARE @Id INT, @Value DECIMAL;
5 - DECLARE cursor_data CURSOR FOR
6 - SELECT ID, Value FROM SourceTable;
7 OPEN cursor_data;
8 - FETCH NEXT FROM cursor_data INTO @Id, @Value;
8 WHILE @@FETCH_STATUS = 0
9 BEGIN
10 - -- Process row-by-row
11 - UPDATE TargetTable
12 - SET ProcessedValue = @Value * 1.1
12 - WHERE ID = @Id;
13 - FETCH NEXT FROM cursor_data INTO @Id, @Value;
14 END;
15 CLOSE cursor_data;
16 DEALLOCATE cursor_data;
16 END;
```

Diff View

✓ Accept

✗ Reject

Generated PySpark

```
1 + # Modern PySpark Functional
2 + from pyspark.sql import SparkSession
3 + from pyspark.sql.functions import col
4
5 + def process_data_functional(spark: SparkSession):
6 +     source_df = spark.read.table("SourceTable")
7
8 +     # Vectorized operation
9 +     processed_df = source_df.withColumn(
10         "ProcessedValue",
11         col("Value") * 1.1
12     )
13
14 +     processed_df.write.mode("overwrite").saveAsTable("TargetTable")
15
16     # Execution is parallel & distributed
```

Selector de Modelos:

El usuario decide qué cerebro usar para cada tarea.

Side-by-Side Diff:

Comparación visual inmediata: Legacy vs. Cloud.

Aceptar/Rechazar:

El usuario tiene la última palabra. Se pueden aplicar mejoras específicas (ej. 'Forzar tipado', 'Optimizar joins').

Estadio 4: Gobernanza y Salida

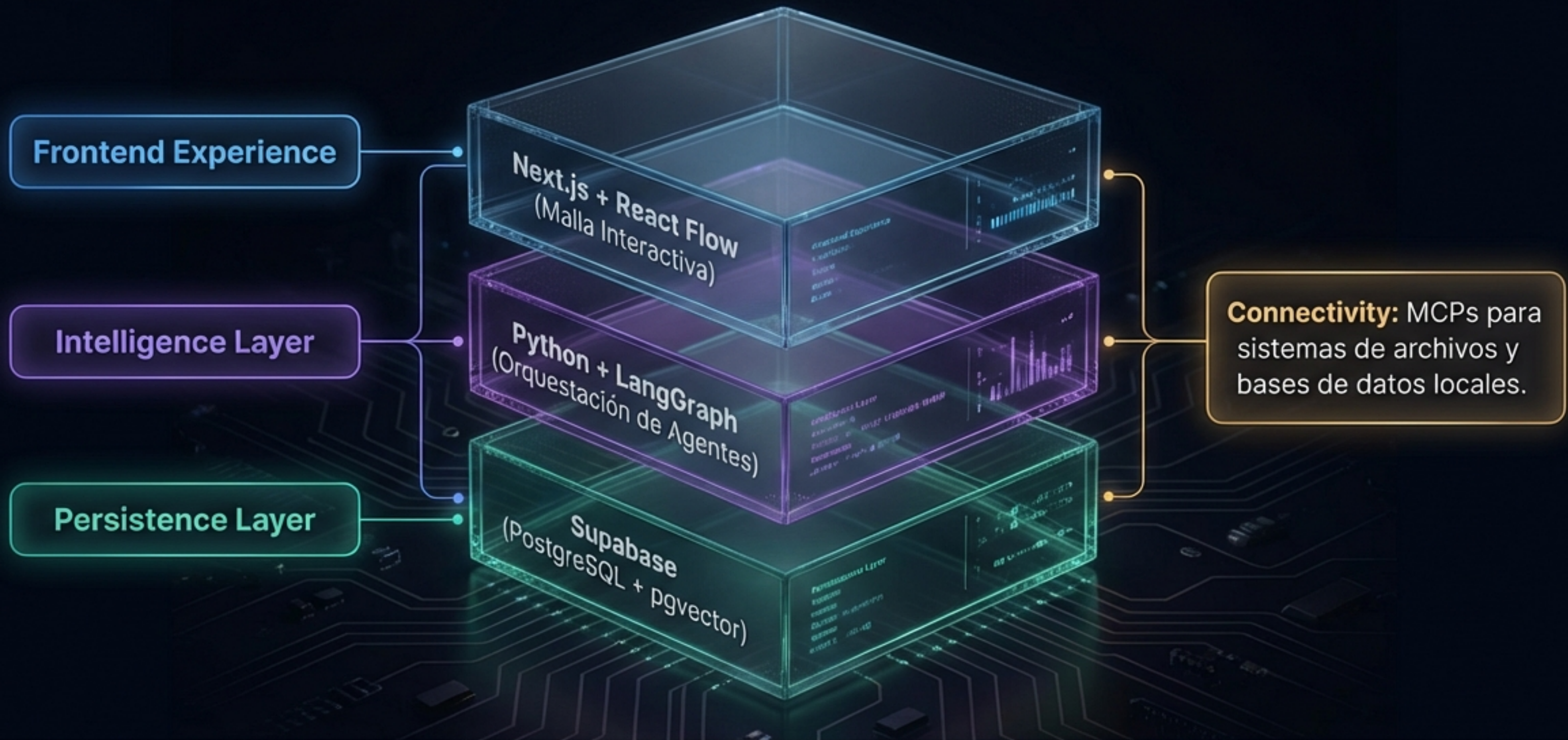
Un repositorio listo para producción, documentado y gobernable.



- Repositorio Limpio: Código PySpark/SQL organizado y ejecutable.
- ✓ Metadata Técnica: Dicionarios de datos generados automáticamente.
- Linaje Visual: Mapas de trazabilidad compatibles con Apache Atlas/Informatica.
- ✓ Integración: Listo para despliegue en Databricks, Cloudera o Snowflake.

Arquitectura Técnica: Robustez Empresarial

Tech Stack



Impacto Real: Caso Helvetia Caser

Consolidación de la Enterprise Data Platform.

350

Tablas SQL Server Migradas

102

Flujos de Orquestación SAP IQ

Módulos Críticos

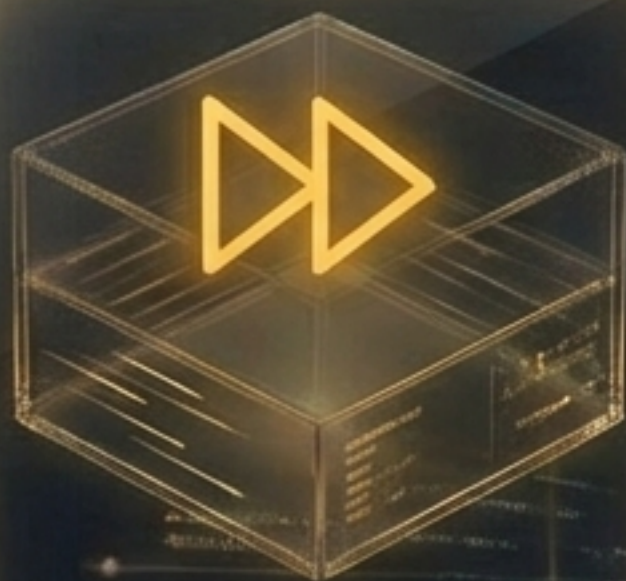
Clientes, Siniestros y Pólizas

Resultado

Apache Iceberg v2 (ACID)

Reducción drástica del tiempo de desarrollo vs. reescritura manual.

Por qué 'Mover la T' con Shift-T



Velocidad

Aceleración exponencial de migraciones complejas.



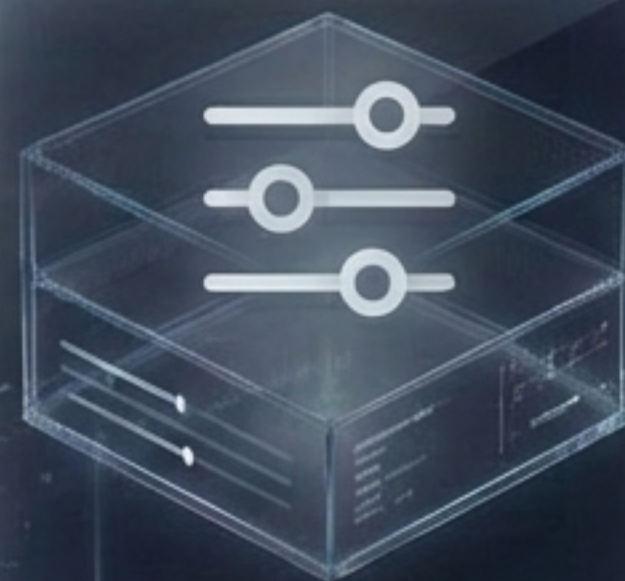
Calidad

Código nativo de nube, no traducciones literales.



Conocimiento

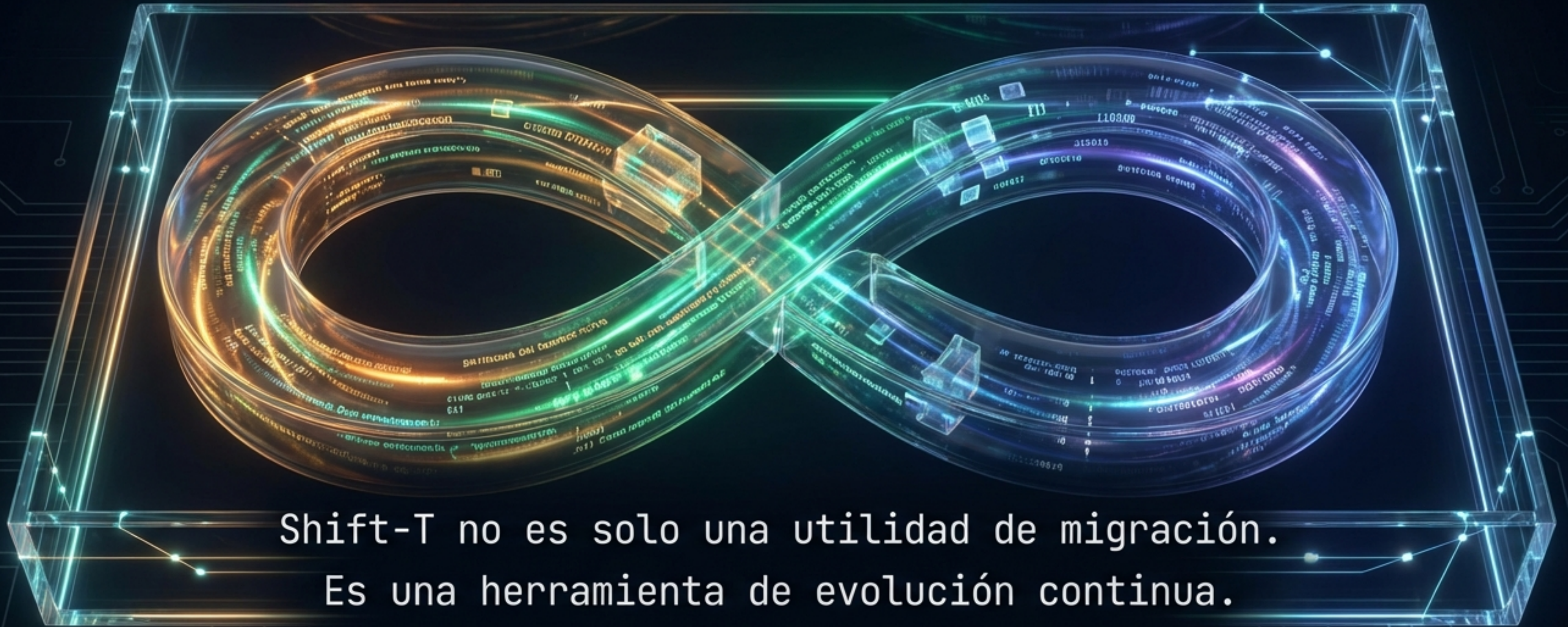
Documentación y linaje donde antes había cajas negras.



Control

La IA propone, el experto humano dispone.

Futuro del Desarrollo



Shift-T no es solo una utilidad de migración.

Es una herramienta de evolución continua.

Es una herramienta de evolución continua.

Posicionamiento hacia Data Governance y optimización perpetua.



**Libera tus datos del legado.
Mueve la T.**

[GitHub Repo / Website URL]